

Python OOP documentation (Object-oriented-programming)

Create an object (**class** function)

```
class AuditReport: (remember the semicolons and first letter capital in class function)
```

```
    def __init__ (self, report_title, auditor):
```

```
        self.report_title = report_title
```

```
        self.auditor = auditor
```

Dot notation

Dot notation is to access the init method

The `__init__` method is to give an object's self, attributes and their behavior

Here in init method `report_title`, `auditor` is the attributes of the superclass (AuditReport)

And with `self`. `Report_title = report_title`, you can access its methods

Here are the methods:

```
audit_report_A = AuditReport("MNU Report", "Mohammed Raffan")
```

this audit report auditor is Mohammed Raffan, and the title is MNU Report

see how I access to methods I have given to audit report attributes by using Dot notation

```
print (audit_report_A.report_title)
```

Report_title is an attribute, and audit_report_A has the values

This is how you create an Object and give attributes and methods

```
class AuditReport:
```

```
    def __init__(self, report_title, auditor):
```

```
        self.report_title = report_title
```

```
        self.auditor = auditor
```

```
audit_report_A = AuditReport("MNU Report", "Mohammed Raffan")
```

Output:

```
print(audit_report_A.report_title)
```

```
MNU Report
```

Now let's create another class or another Object. But this time, we won't mention detail that our audit report needs like title, auditor. Instead, we use the `super()` function to call superclass methods. Here, `AuditReport` is our superclass so create a new audit report

```
MNU_Audit_Rep2 = AuditReport("MNU Opportunities","William Jack")
```

This time it looks a bit different than the first one, but have to mention the superclass and fill the attributes that you have mentioned in the superclass, that's all you have to do

And print it to get the output. Use the Dot notation

```
print (MNU_Audit_Rep2.report_title)
```

Now we can access to our new class or subclass with the same attributes we gave but will have different behavior or different methods

Output:

```
print(MNU_Audit_Rep2.report_title)
```

```
MNU Opportunities
```

```
class AuditReport:
```

```
    def __init__(self, report_title, auditor):
```

```
        self.report_title = report_title
```

```
        self.auditor = auditor
```

```
audit_report_A = AuditReport("MNU Report", "Mohammed Raffan")
```

```
MNU_Audit_Rep2 = AuditReport("MNU Opportunities","William Jack")
```

```
print(MNU_Audit_Rep2.report_title)
```

Output: MNU Opportunities
